

Sơ bộ

1. Yêu cầu của Thầy

Hiểu bài toán, mô tả được bài toán / vấn đề.

Hiểu đầu vào – đầu ra của bài toán.

Hiểu dữ liệu dùng (nguồn, số lượng, class, đặc điểm, cách chia train/val/test).

Hiểu mô hình sử dụng, đặc trưng của mô hình (EfficientNet-B0 + các kỹ thuật liên quan).

Mô tả quy trình giải bài toán (pipeline từ data → preprocess → model → train → eval).

Có bảng kết quả hoặc hình vẽ kết quả.

Có nhận xét / phân tích kết quả.

Có hướng phát triển tiếp theo.

Nộp: slide + report PDF (đầy đủ các phần trên).

2. Mình làm thêm

Có phần lý thuyết nền (skin cancer, CNN, EfficientNet, imbalance, TTA, MC Dropout...).

Có mô tả kỹ về cách cân bằng dataset (augmentation + downsampling).

Có mô tả chi tiết mô hình & pipeline (kiến trúc, flowchart, sequence diagram nếu được).

Có đánh giá nhiều chỉ số: accuracy, precision, recall, F1, confusion matrix, có thể thêm ROC/AUC.

Có so sánh baseline vs mô hình cải tiến (ví dụ: chỉ fine-tune đơn giản vs thêm TTA/MC Dropout).

Colab

Nguyên: 1, 2, 10

Phát: 3, 4, 5, 6

Đức: 7, 8, 9

Khởi công việc code (trên Google Colab)

Tóm gọn checklist:

1. Tạo notebook Colab:
 - Mount Google Drive (nếu cần).
 - Tải HAM10000 từ Kaggle (API hoặc upload).
2. EDA + cân bằng dữ liệu:
 - Đọc metadata CSV.
 - Merge ảnh + label.
 - Thống kê class.
 - Viết code augment + downsample để tạo dataset 7512 ảnh. (Minority classes (akiec, bcc, df, vasc) được augment đến 1000 ảnh; majority nv downsample còn 1300; bkl, mel giữ nguyên. Tổng 7512 ảnh.)
 - Lưu lại phân bố sau khi cân bằng.

3. Chuẩn bị pipeline dữ liệu (tf.data hoặc ImageDataGenerator)

3.1. Encode label

- Tạo mapping class → index (ổn định, ghi lại để dùng khi vẽ confusion matrix):

python

```
class_names = ['akiec', 'bcc', 'bkl', 'df', 'mel', 'nv', 'vasc']  
class_to_idx = {c: i for i, c in enumerate(class_names)}
```

- Thêm cột label_idx cho df_balanced.

3.2. Chia train/val/test (stratified)

- Dùng `train_test_split` 2 lần:
 - Đầu tiên: train_val (85%) & test (15%) với `stratify=df_balanced['dx']`
 - Sau đó: từ train_val → train (70%) & val (15%) theo tỉ lệ 70/15/15 tổng thể (stratify tiếp).
- Kết quả 3 DataFrame: `df_train`, `df_val`, `df_test`
- Print `value_counts()` của mỗi set để confirm.

3.3. Viết hàm load & preprocess ảnh

- Chuẩn hóa như paper:
 - Resize 224×224
 - Normalize bằng **ImageNet mean/std** (tương đương dùng layer `tf.keras.applications.efficientnet.preprocess_input`).

```
IMG_SIZE = 224  
  
def load_image(path, label):  
    img = tf.io.read_file(path)  
    img = tf.image.decode_jpeg(img, channels=3)  
    img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])  
    img = tf.keras.applications.efficientnet.preprocess_input(img)  
    return img, label
```

3.4. Tạo `tf.data.Dataset`

- Convert list path + labels từ `df_train/val/test` sang `tf.data.Dataset`:

```
train_ds = tf.data.Dataset.from_tensor_slices((train_paths, train_labels))
train_ds = train_ds.map(load_image, num_parallel_calls=AUTOTUNE)
train_ds = train_ds.shuffle(...).batch(BATCH_SIZE).prefetch(AUTOTUNE)
```

- Với test, thêm augmentation on-the-fly (flip, rotation, zoom...) bằng `tf.keras.layers.RandomFlip`, `RandomRotation`, etc.

4. Xây EfficientNet-B0

4.1. Định nghĩa mô hình cơ bản

- Load backbone:

```
base_model = tf.keras.applications.EfficientNetB0(
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    weights='imagenet'
)
base_model.trainable = False # Phase 1
```

- Thêm head:

```
inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = base_model(inputs, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.3)(x)
outputs = tf.keras.layers.Dense(len(class_names), activation='softmax')(x)
model = tf.keras.Model(inputs, outputs)
```

4.2. Compile (Phase 1)

- Optimizer: Adam
- Loss: `SparseCategoricalCrossentropy`

- Metrics: ['accuracy']

5. Huấn luyện – Phase 1 (freeze base, train classifier)

5.1. Callbacks

- `ModelCheckpoint` (save best val acc)
- `EarlyStopping` (monitor val_loss, patience)
- `ReduceLROnPlateau` (optional)

5.2. Train

- `history1 = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS1, callbacks=...)`
- Log kết quả:
 - Vẽ đồ thị train/val accuracy & loss
 - Nhận xét ngắn: độ hội tụ, overfit hay chưa.

6. Huấn luyện – Phase 2 (fine-tune toàn bộ EfficientNet-B0)

Theo paper: unfreeze toàn bộ EfficientNet-B0, lr nhỏ hơn ($1e-4$).

6.1. Unfreeze base_model

```
base_model.trainable = True
for layer in base_model.layers:
    layer.trainable = True
```

6.2. Compile lại với lr nhỏ

- `optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)`
- Compile lại model.

6.3. Train Phase 2

- `history2 = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS2, callbacks=...)`
- Lưu best weights (nếu đang dùng ModelCheckpoint).
- Vẽ đồ thị Phase 2, nhận xét: Accuracy tăng, val acc ~95% như paper.

7. Đánh giá cơ bản trên test set

7.1. Predict trên test_ds

- Tạo `test_ds` giống train/val nhưng `shuffle=False`.
- `y_prob = model.predict(test_ds)`
`y_pred = np.argmax(y_prob, axis=1)`
`y_true` lấy từ `df_test['label_idx']`.

7.2. Metrics

- `accuracy_score(y_true, y_pred)`
- `classification_report` (precision, recall, F1 cho từng lớp).
So sánh với bảng 4 trong paper (precision/recall từng lớp, macro/weighted).

7.3. Confusion matrix

- Tính `confusion_matrix`.

- Vẽ heatmap với `seaborn.heatmap`, tick bằng `class_names`.
- Lưu hình (dùng sau cho report PDF).

7.4. ROC/AUC (nếu làm multi-class)

- Convert labels one-hot.
- Dùng `roc_curve`, `auc` cho từng lớp.
- Vẽ multi-class ROC (optional, nhưng rất đẹp để bỏ vào report, giống Figure 6 trong paper).

8. TTA + MC Dropout (phần “ngẫu”, bám Algorithm 1)

Paper có **Algorithm 1: Skin Lesion Image Classification using EfficientNet-B0 with Enhanced TTA and MC Dropout**. Ta sẽ code lại idea đó.

8.1. Kích hoạt dropout lúc inference (MC Dropout)

- Viết hàm `enable_dropout(model)`:
 - Iterate các layer; nếu là `Dropout`, set `layer.trainable = True` và ép call với `training=True`.
 - Hoặc đơn giản hơn: trong hàm `predict custom`, gọi `model(x, training=True)`.

8.2. Định nghĩa tập augmentation TTA

Theo paper: flip ngang, dọc, color jitter, rotation $\pm 20^\circ/45^\circ$ (ở phần mô tả có cả 20° và $\pm 45^\circ$).

- Viết các hàm augmentation image-level:
 - Identity (original)
 - Horizontal flip
 - Vertical flip
 - Small rotation

- Color jitter (brightness/contrast đổi nhẹ)
- TTA list: [aug_none, aug_hflip, aug_vflip, aug_rotate, aug_color]

8.3. Hàm inference TTA + MC Dropout

Hàm như pseudo code Algorithm 1 (paper):

```
def tta_mc_predict(model, images, M=10):
    # images: tensor (N, H, W, 3) đã preprocess
    all_probs = [] # sẽ chứa các dự đoán softmax

    for aug in aug_list:
        x_aug = aug(images)
        for m in range(M):
            probs = model(x_aug, training=True) # MC dropout
            all_probs.append(probs.numpy())

    all_probs = np.stack(all_probs, axis=0) # (A*M, N, C)
    mean_probs = all_probs.mean(axis=0) # (N, C)
    y_pred = mean_probs.argmax(axis=1)
    y_conf = mean_probs.max(axis=1)
    return y_pred, y_conf, mean_probs
```

- Với test set, có thể chạy theo batch để tránh dùng quá nhiều RAM.

8.4. So sánh performance

- Chạy:
 - Baseline: dùng `model.predict` bình thường → test accuracy, F1.
 - TTA + MC: dùng `tta_mc_predict` → test accuracy, F1.
- In ra bảng so sánh:
 - Accuracy, Macro F1, Weighted F1.
- Nhận xét: TTA + MC dropout giúp tăng nhẹ accuracy (gần 97.15% như paper) và ổn định hơn.

9. Lưu model + xuất hình kết quả

9.1. Lưu model/weights

- `model.save(f'{BASE_DIR}/efficientnet_b0_ham10000.h5')`
- Hoặc lưu chỉ weights: `model.save_weights(...)`

9.2. Lưu các hình phục vụ report PDF

- Đồ thị train/val loss & acc (Phase 1 + Phase 2).
 - Confusion matrix.
 - ROC curves (nếu có).
- Có thể lưu vài ảnh test kèm dự đoán & confidence (để minh họa trong report).

10. (Optional) Demo web / simple API

Nếu nhóm muốn bám theo paper (họ còn deploy web-based hệ thống phân loại).

- Viết một cell tạo hàm `predict_single_image(path)`:
 - Load ảnh, preprocess, gọi TTA + MC, trả về label + confidence.
- Có thể dùng Gradio / Streamlit cho demo (không bắt buộc với môn học, nhưng nếu còn thời gian thì rất đẹp).

PDF

I. TÓM TẮT & GIỚI THIỆU (Nguyên)

1.1. Giới thiệu Vấn đề

- **Vấn đề:** Ung thư da là một trong những loại ung thư phổ biến và gây tử vong cao nhất trên toàn cầu.
- **Nhu cầu:** Cần có phương pháp chẩn đoán sớm và chính xác là rất cần thiết cho việc điều trị hiệu quả.
- **Thách thức:** Phương pháp chẩn đoán truyền thống tốn thời gian, đòi hỏi chuyên môn cao, và khó mở rộng. Các kỹ thuật Học sâu (DL) hiện nay gặp khó khăn về mất cân bằng dữ liệu, hiệu quả tính toán, và độ tin cậy của dự đoán.

1.2. Mục tiêu Nghiên cứu

- **Mục tiêu chính:** Phát triển một khung phân loại tổn thương da đa lớp hiệu quả và nhẹ sử dụng EfficientNet-B0.
- **Mục tiêu kỹ thuật:** Giảm thiểu mất cân bằng dữ liệu bằng kỹ thuật tăng cường (augmentation) và giảm mẫu (downsampling). Tăng cường độ mạnh mẽ và độ chính xác bằng cách tích hợp **Transfer Learning**, **Fine-Tuning**, **Test-Time Augmentation (TTA)**, và **Monte Carlo Dropout (MC Dropout)**.
- **Đóng góp cốt lõi:**
 - Phát triển mô hình EfficientNet-B0 được tùy chỉnh cho 7 loại tổn thương da.
 - Xây dựng bộ dữ liệu cân bằng (7512 ảnh) từ HAM10000.
 - Đạt độ chính xác đỉnh cao **97.15%** trên tập kiểm tra.
 - Triển khai mô hình trên nền tảng hỗ trợ quyết định lâm sàng dựa trên web.

1.3. Lý thuyết Nền

- **Tổn thương da & Phân loại:** Mô tả 7 loại tổn thương cần phân loại: **akiec**, **bcc**, **bkl**, **df**, **mel**, **nv**, **vasc**.
- **CNN và Deep Learning:** Vai trò của Mạng nơ-ron tích chập trong phân tích hình ảnh y tế.
- **EfficientNet-B0:** Đặc điểm của kiến trúc này (nhẹ và mạnh mẽ), được chọn vì sự cân bằng tối ưu giữa độ chính xác và hiệu quả tính toán.
- **Transfer Learning & Fine-Tuning:** Kỹ thuật tận dụng các tính năng đã được học từ ImageNet.
- **TTA & MC Dropout:** Các kỹ thuật suy luận tăng cường giúp tăng độ mạnh mẽ và định lượng sự không chắc chắn (uncertainty quantification).

II. PHƯƠNG PHÁP VÀ DỮ LIỆU (Phát)

2.1. Dữ liệu Sử dụng

- **Nguồn Dữ liệu:** Bộ dữ liệu **HAM10000** (Human Against Machine).
- **Số lượng & Đặc điểm:** Tổng cộng **10,015** hình ảnh dermoscopic ban đầu, chia thành 7 danh mục chẩn đoán.
 - **Hình ảnh minh họa:** **Figure 3** (Các mẫu dữ liệu cho 7 lớp).
- **Tiền xử lý & Cân bằng Dữ liệu:**
 - **Vấn đề Mất cân bằng:** Dữ liệu ban đầu bị mất cân bằng lớp.

- **Cân bằng lớp:** Bộ dữ liệu được cân bằng lại thành **7512** hình ảnh.
 - **Augmentation (Tăng cường):** Các lớp thiếu số (**akiec, bcc, df, vasc**) được tăng cường (rotation, zoom, shifts, flips) để đạt **1000** hình ảnh mỗi lớp.
 - **Downsampling (Giảm mẫu):** Lớp đa số **nv** được giảm mẫu xuống còn **1300** hình ảnh.
- **Chia Tập Dữ liệu:** Sử dụng lấy mẫu phân tầng (stratified sampling) để đảm bảo tỷ lệ lớp đồng đều.
 - Tỷ lệ: **70% Huấn luyện (5258 ảnh), 15% Kiểm định (1127 ảnh), 15% Kiểm tra (1127 ảnh).**
- **Chuẩn hóa:** Tất cả hình ảnh được thay đổi kích thước thành **224×224 pixels** và chuẩn hóa bằng thống kê ImageNet.

2.2. Kiến trúc Mô hình và Pipeline

- **Mô hình Cơ sở: EfficientNet-B0**, tiền huấn luyện trên ImageNet.
- **Quy trình Tổng thể (Pipeline):** Được mô tả qua sơ đồ luồng.
 - **Hình minh họa: Figure 1** (Kiến trúc đề xuất và luồng công việc).

2.3. Quy trình Huấn luyện Mô hình

- **Giai đoạn 1: Transfer Learning (Chuyển giao Học)**
 - **Thiết lập:** Các lớp tích chập (convolutional base) của EfficientNet-B0 được **đóng băng (frozen)**. Chỉ huấn luyện lớp phân loại mới (classification head) cho 7 lớp.
 - **Tham số:** Tối ưu hóa Adam, hàm mất mát CrossEntropyLoss, 15 epochs.
- **Giai đoạn 2: Fine-Tuning (Tinh chỉnh)**
 - **Thiết lập:** Tất cả các lớp của EfficientNet-B0 được **mở khóa (unfrozen)**.
 - **Tham số:** Tiếp tục huấn luyện 15 epochs với tốc độ học tập giảm (1×10^{-4}).

2.4. Quy trình Suy luận Tăng cường (TTA & MC Dropout Inference)

(Thêm: Chi tiết Pipeline)

- **Mô tả Kỹ thuật:** Để tăng cường độ tin cậy (robustness), mô hình sử dụng hai nhánh suy luận bổ sung.
 - **MC Dropout:** Giữ các lớp Dropout **hoạt động (train mode)** trong quá trình suy luận và thực hiện **10 lượt chuyển tiếp ngẫu nhiên ($M=10$)** để định lượng độ không chắc chắn (uncertainty) và tính toán dự đoán trung bình.
 - **TTA (Test-Time Augmentation):** Áp dụng 5 phép biến đổi xác định (góc, lật ngang, lật dọc, Color Jitter, xoay 20°) lên ảnh đầu vào.
 - **Hình minh họa: Figure 4** (Ví dụ về các phép biến đổi TTA).
 - **Hợp nhất (Ensemble):** Kết quả (xác suất Softmax) từ cả TTA và MC Dropout được tính trung bình để đưa ra Dự đoán Cuối cùng và Điểm Độ tin cậy (Confidence Score).
- **Thuật toán & Luồng chi tiết: Algorithm 1** mô tả chi tiết quy trình TTA và MC Dropout. **Figure 2** là Sơ đồ Trình tự (Sequence Diagram) cho quy trình phân loại ảnh đơn lẻ.

III. KẾT QUẢ VÀ THẢO LUẬN (RESULTS AND DISCUSSION) (Nguyên vs Đức)

3.1. So sánh Baseline và Hiệu suất Huấn luyện (Thêm: So sánh Baseline)

• So sánh Kiến trúc CNN (Trước Fine-Tuning):

◦ **Bảng Kết quả: Table 2** (So sánh độ chính xác mô hình trước khi tinh chỉnh).

◦ **Phân tích:** EfficientNet-B0 được chọn vì đạt độ chính xác cao nhất (**89.36%** với tỉ lệ chia 70:15:15), vượt trội hơn ResNet50 (85.42%) và DenseNet121 (86.17%) trong giai đoạn baseline.

• Kết quả Transfer Learning:

◦ **Bảng Kết quả: Table 1** (Hiệu suất huấn luyện và kiểm định với lớp cơ sở đóng băng).

◦ **Phân tích:** Độ chính xác kiểm định tăng từ 64.10% (Epoch 1) lên **73.40%** (Epoch 15).

• Kết quả Fine-Tuning:

◦ **Bảng Kết quả: Table 3** (Hiệu suất huấn luyện và kiểm định trong quá trình tinh chỉnh toàn mạng).

◦ **Phân tích:** Độ chính xác huấn luyện đạt 99.08%, và độ chính xác kiểm định đạt **89.36%** (Sau 15 epochs Fine-Tuning).

3.2. Đánh giá Tập Kiểm tra Cuối cùng (Test Set Evaluation)

• So sánh Cải tiến (Thêm: So sánh Mô hình Cải tiến):

◦ Mô hình Fine-Tuned đơn thuần: 95%.

◦ Fine-Tuned + TTA: 96%.

◦ Fine-Tuned + TTA + MC Dropout (Mô hình Cuối cùng): **97.15%**.

• **Độ chính xác Tổng thể:** Độ chính xác cuối cùng trên tập kiểm tra (1127 mẫu) là **97.15%**.

3.3. Phân tích Chi tiết Hiệu suất (Classification Metrics) (Thêm: Đánh giá Nhiều Chỉ số)

• Báo cáo Phân loại:

◦ **Bảng Kết quả: Table 4** (Báo cáo phân loại tóm tắt Precision, Recall, F1-Score).

◦ **Nhận xét:** Đạt Precision và Recall hoàn hảo **1.00** cho các lớp **df** và **vasc**. Các chỉ số F1-Score trung bình Macro và Weighted đều đạt xấp xỉ **0.97**, cho thấy hiệu suất cân bằng giữa các lớp đa số và thiểu số.

• Ma trận Nhầm lẫn (Confusion Matrix):

◦ **Hình minh họa: Figure 5** (Ma trận nhầm lẫn).

◦ **Nhận xét:** Ma trận xác nhận khả năng phân loại mạnh mẽ, với các giá trị chẩn đoán đúng trên đường chéo cao. Các lớp **bkl** và **mel** có F1-score thấp hơn một chút, có thể do sự chồng chéo hoặc tương đồng hình ảnh.

• Đường cong ROC và AUC:

◦ **Hình minh họa: Figure 6** (Phân tích AUC-ROC).

- **Nhận xét:** Mô hình cho thấy khả năng phân biệt lớp xuất sắc, với **tất cả các lớp đều đạt AUC = 1.00**, bao gồm cả micro-average và macro-average, khẳng định độ mạnh mẽ của mô hình trong việc tách biệt các loại tổn thương.

3.4. Demo hệ thống Web (Web-Based System) (Yêu cầu của Thầy) (Đức)

- **Mục đích:** Đánh giá tính ứng dụng thực tế của mô hình EfficientNet-B0 trong môi trường lâm sàng.

- **Chức năng:** Hệ thống web cung cấp phân loại tự động, điểm độ tin cậy và hiển thị hình ảnh.

- **Ví dụ:**

- **Figure 7:** Hệ thống dự đoán **Benign Keratosis (bkl)** với độ tin cậy **99.95%**.

- **Figure 8:** Hệ thống dự đoán **Melanoma (mel)** với độ tin cậy **99.70%**.

- **Phân tích:** Nền tảng này hỗ trợ các quyết định lâm sàng bằng cách phân biệt tin cậy giữa các tổn thương lành tính và ác tính, giúp ưu tiên các trường hợp rủi ro cao.

IV. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN TIẾP THEO (Đức)

4.1. Kết luận (Conclusions)

- Nghiên cứu đã phát triển thành công khung **EfficientNet-B0** để phân loại đa lớp ung thư da.

- Chiến lược tiền xử lý dữ liệu và cân bằng lớp đã khắc phục vấn đề mất cân bằng dữ liệu.

- Việc tích hợp **Transfer Learning, Fine-Tuning, MC Dropout** và **TTA** đã đạt được độ chính xác cuối cùng là **97.15%**.

- Do thiết kế nhẹ và hiệu suất mạnh mẽ, EfficientNet-B0 là một ứng viên phù hợp để triển khai trong môi trường lâm sàng với tài nguyên tính toán hạn chế.

4.2. Hướng Phát triển Tiếp theo (Future Work) (Yêu cầu của Thầy)

- Nghiên cứu trong tương lai sẽ tập trung vào việc khám phá **các phương pháp siêu phân giải nhẹ (lightweight super-resolution methods)**.

- Cải thiện **định lượng độ không chắc chắn** để tăng cường độ mạnh mẽ và khả năng áp dụng lâm sàng.

Slide

1. Title + thông tin nhóm.
2. Đặt vấn đề & thách thức thực tế
3. Mục tiêu & Input/Output
4. Dataset HAM10000
5. Xử lý mất cân bằng + chia tập
6. Preprocess & Augmentation
7. Vì sao EfficientNet-B0?
8. Huấn luyện 2 giai đoạn
9. Key Innovation khi suy luận
10. Kết quả huấn luyện (learning progress)
11. Kết quả Test & so sánh baseline (BẢNG)
12. Phân tích chi tiết lỗi
13. Demo web
14. Kết luận + Hạn chế & Hướng phát triển

MỤC LỤC BÀI THUYẾT TRÌNH

I. MỞ ĐẦU (1.5 phút)

- Giới thiệu vấn đề:** Tính cấp thiết của việc chẩn đoán sớm ung thư da và những thách thức trong thực tiễn y tế (mất cân bằng dữ liệu, độ tin cậy dự đoán).
- Mục tiêu đề tài:** Xây dựng khung phân loại đa lớp dựa trên **EfficientNet-B0** tích hợp các kỹ thuật tăng cường độ vững chắc.

II. DỮ LIỆU & QUY TRÌNH TIỀN XỬ LÝ (2 phút)

- Dữ liệu HAM10000:** Đặc điểm 7 lớp tổn thương da và thực trạng mất cân bằng dữ liệu ban đầu.
- Chiến lược cân bằng dữ liệu:** Kết hợp **Oversampling** (cho lớp thiểu số) và **Undersampling** (cho lớp đa số NV).
- Tiền xử lý & Augmentation:** Chuẩn hóa ảnh về 224x224 và các phép biến đổi ngẫu nhiên (xoay, lật, zoom, tương phản) để tăng tính đa dạng cho mô hình.

III. PHƯƠNG PHÁP & KIẾN TRÚC MÔ HÌNH (2 phút)

- Kiến trúc EfficientNet-B0:** Lý do lựa chọn dựa trên sự tối ưu giữa hiệu năng và chi phí tính toán (khối MBConv, cơ chế SE, hàm Swish).
- Quy trình huấn luyện 2 giai đoạn:**
 - Transfer Learning:** Đóng băng backbone để thích nghi lớp phân loại mới.
 - Fine-tuning:** Giải băng toàn mạng để tinh chỉnh sâu các đặc trưng ảnh y tế.
- Kỹ thuật suy luận nâng cao:** Tăng độ ổn định và định lượng độ bất định thông qua **TTA (Test-Time Augmentation)** và **MC Dropout**.

IV. KẾT QUẢ & PHÂN TÍCH (3.5 phút)

- Hiệu suất huấn luyện:** Sự cải thiện độ chính xác trước và sau khi thực hiện Fine-tuning.
- Đánh giá trên tập Kiểm tra (Test set):**
 - So sánh hiệu năng giữa EfficientNet-B0 và các mô hình baseline (ResNet50, MobileNetV2...) khi áp dụng TTA.
 - Phân tích chi tiết các chỉ số **Accuracy (87.49%)**, **F1-Score** theo từng lớp.
- Phân tích sai số & Độ tin cậy:**
 - Ma trận nhầm lẫn (Confusion Matrix):** Nhận diện các cặp lớp dễ gây nhầm lẫn (MEL, AKIEC, BCC).
 - Biểu đồ ROC-AUC:** Khả năng phân tách lớp của mô hình.
- Demo Hệ thống Web:** Giới thiệu ứng dụng thực tế trên nền tảng **Streamlit**, quy trình chẩn đoán và trực quan hóa độ tin cậy (Confidence Score).

V. KẾT LUẬN & HƯỚNG PHÁT TRIỂN (1 phút)

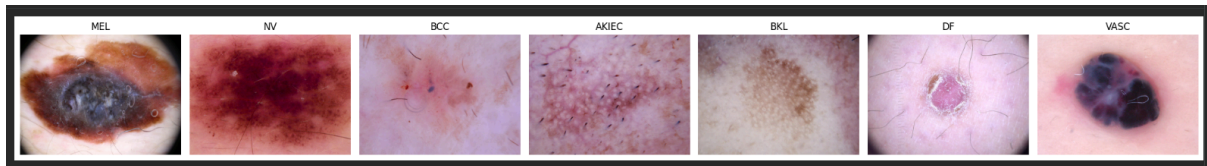
- Tổng kết:** Những thành công trong việc kết hợp EfficientNet-B0 với các kỹ thuật xử lý dữ liệu y tế.
- Hạn chế & Hướng phát triển:**
 - Giải quyết triệt để hơn sự nhầm lẫn giữa các lớp tương đồng.
 - Tích hợp **Siêu phân giải (Super-resolution)** và **Grad-CAM** để tăng tính giải thích cho mô hình trong lâm sàng.

Phát làm content

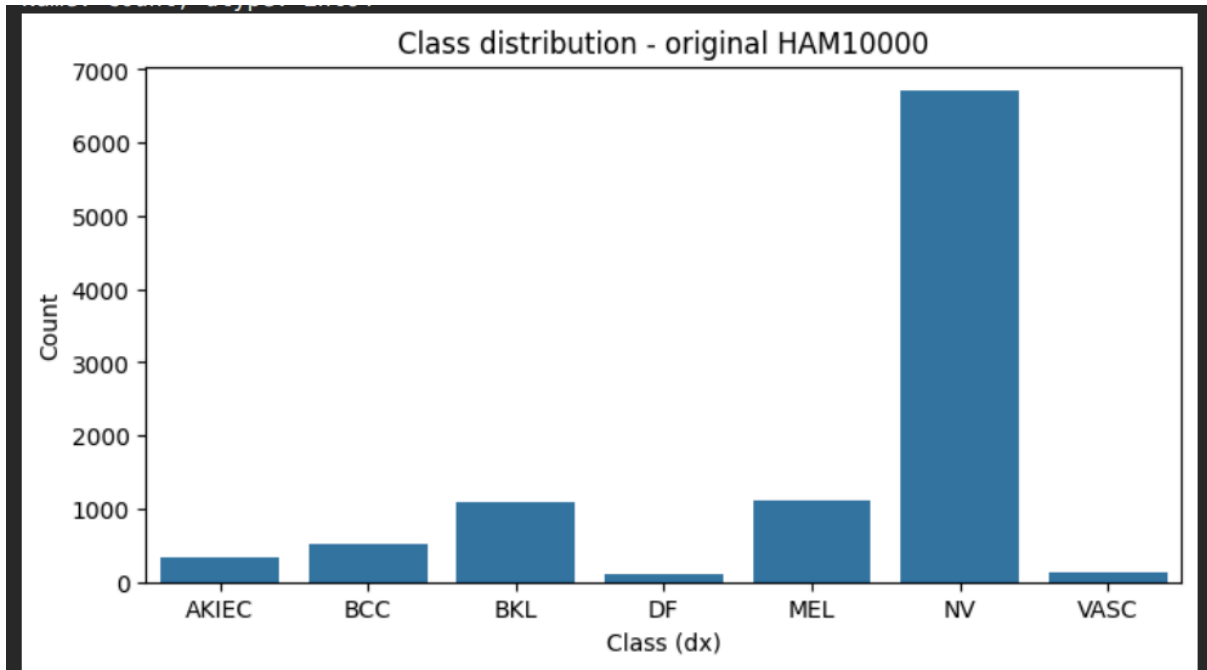
NỘI DUNG CỦA PHÁTs

2.1

-Bộ dữ liệu HAM10000 được sử dụng với 10015 hình ảnh dermoscopic ban đầu chia thành 7 loại bệnh để chẩn đoán là: AKIEC, BCC, BKL, DF, MEL, NV, VASC

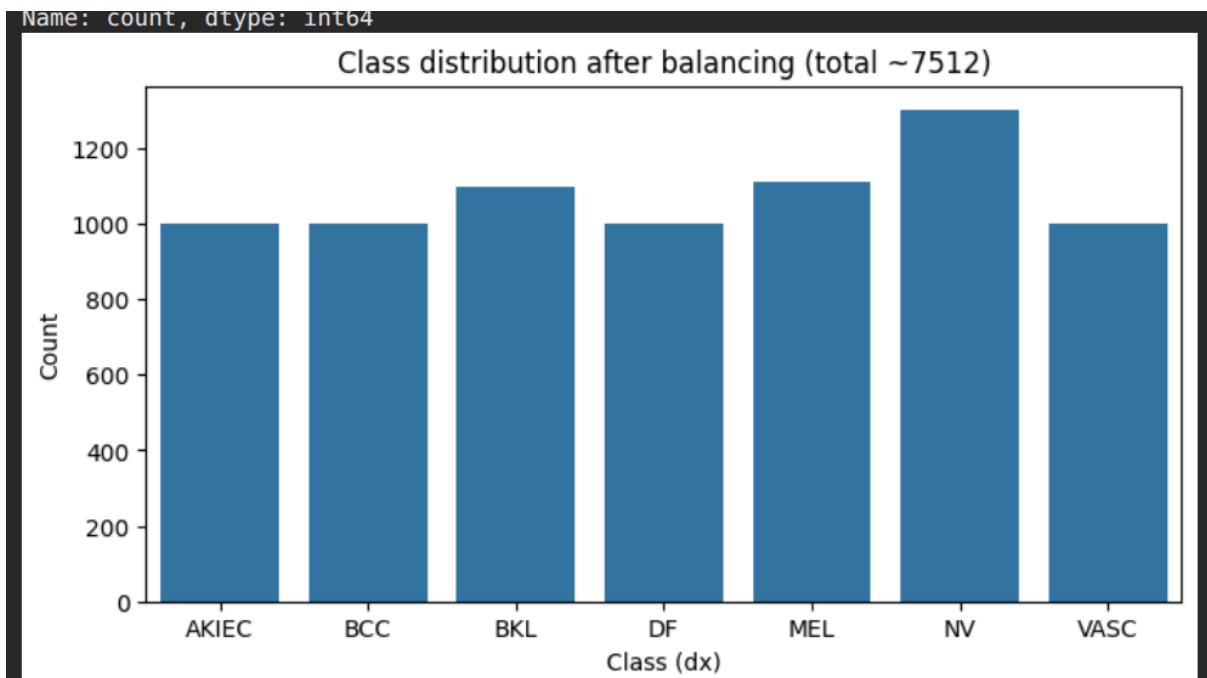


-Dataset ban đầu bị mất cân bằng nghiêm trọng



Do đó ta xử lí mất cân bằng theo quy tắc:

- Các lớp AKIEC, BCC, DF, VASC ta thực hiện oversampling để tăng số lượng mẫu
- Lớp NV có số lượng mẫu cao, ta giảm xuống còn 13000
- BKL, MEL ta giữ nguyên



Số lượng trước và sau khi cân bằng

```
[len(df), len(df_balanced)]
... [10015, 7512]
```

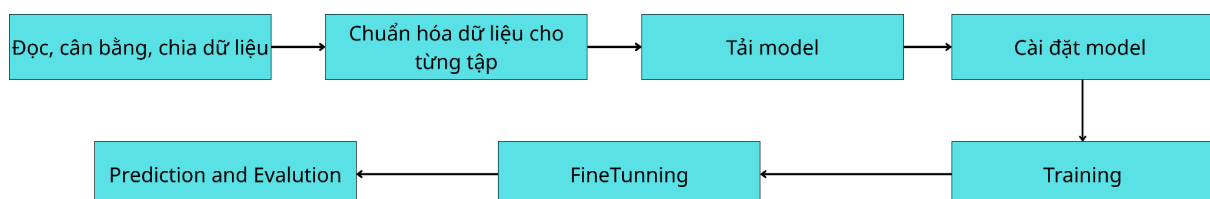
Tiến hành chia dataset đã cân bằng thành 3 tập: train, validation, test với tỉ lệ 70-15-15

```
Train: 5258 dòng
Val: 1127 dòng
Test: 1127 dòng
```

2.2 Kiến trúc mô hình và pipeline

- Sử dụng mô hình EfficientNet-B0 với hình thức TransferLearning
- Dùng MC Dropout để tránh học vẹt trong quá trình training và tăng độ chính xác khi kết hợp với TTA trong quá trình prediction
- Kết hợp TTA trên prediction và Random Augmentation trên training nhằm đa dạng thông tin được học của từng ảnh, tăng tỉ lệ chính xác khi dự đoán

Quy trình tổng thể



Chuẩn hóa dữ liệu.

Vì dữ liệu ban đầu ở dạng đường dẫn đến hình ảnh của bệnh nên phải chuẩn hóa thì máy tính mới hiểu và đưa vào model được, ta chuẩn hóa ảnh về size 224x224

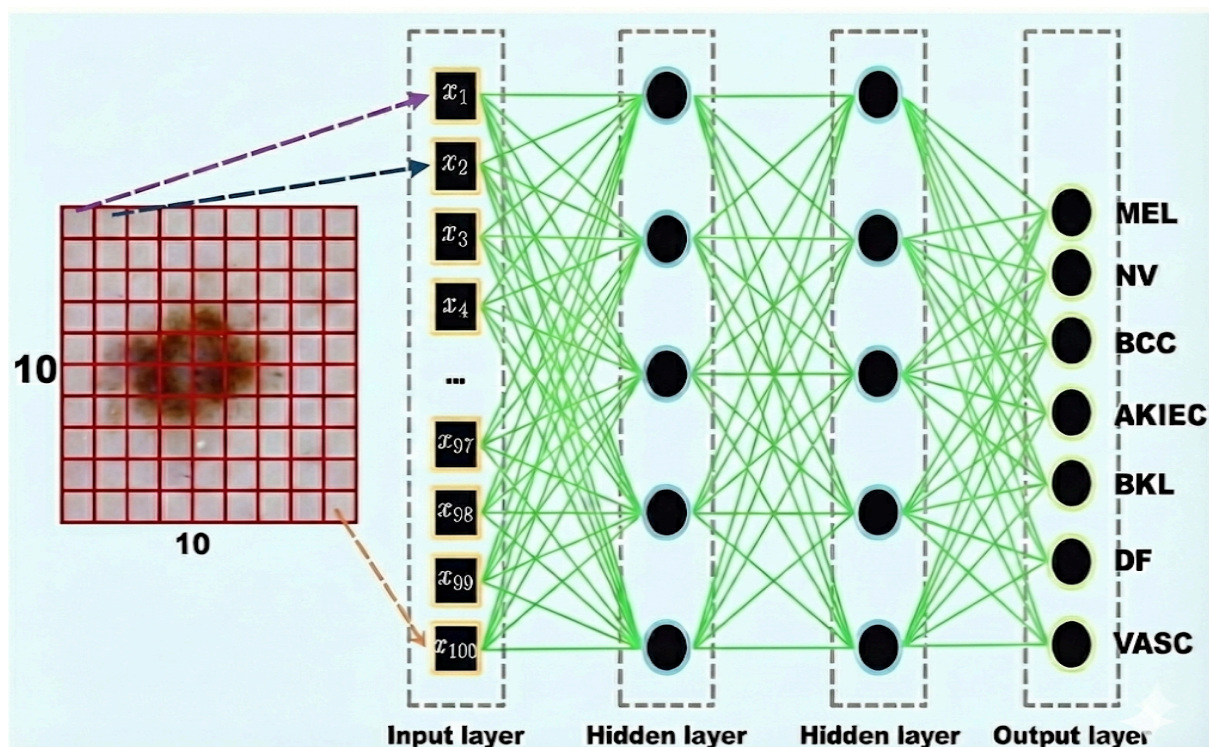
```
▶ IMG_SIZE= 224
def load_image(path, label):
    img= tf.io.read_file(path)
    img= tf.image.decode_jpeg(img,channels=3)
    img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])
    img = tf.cast(img, tf.float32)
    img= tf.keras.applications.efficientnet.preprocess_input(img)
    return img, label
```

Ảnh đưa vào model được xử lí ra sao?

Quá trình training:

- Ảnh ban đầu được đưa vào model sẽ được phân tách ra thành nhiều thành phần hơn, tạo thành input layer
- Sau đó từng đến các layer kế tiếp sẽ trích xuất từng đặc trưng của ảnh
- Cuối cùng nó sẽ đưa ra kết quả dự đoán ở output layer, và chỉnh sửa các trọng số sao cho kết quả đạt được tiệm cận nhất với label tương ứng, tiếp tục quá trình cho đến khi hết dữ liệu train

Quá trình prediction: giống như training nhưng chỉ dự đoán mà không chỉnh sửa tham số



Traning(TransferLearning):

-Là quá trình mà ta để model “hiểu được output mới” bằng cách đóng băng base layer, chỉ giữ các head custom layer (Global Average Pooling 2D, Fully connected Layer, Output layer)

+Global Average Pooling 2D: Mỗi layer sẽ cho trích xuất 1 đặc trưng, do đó cả quá trình sẽ trích ra được rất nhiều đặc trưng (mỗi đặc trưng là 1 ma trận 2D), layer này đảm bảo việc đơn giản hóa tất cả đặc trưng về dạng 1 vectơ duy nhất

+Fully connected Layer: kết nối giữa output và đặc trưng, tăng cường suy luận phi tuyến

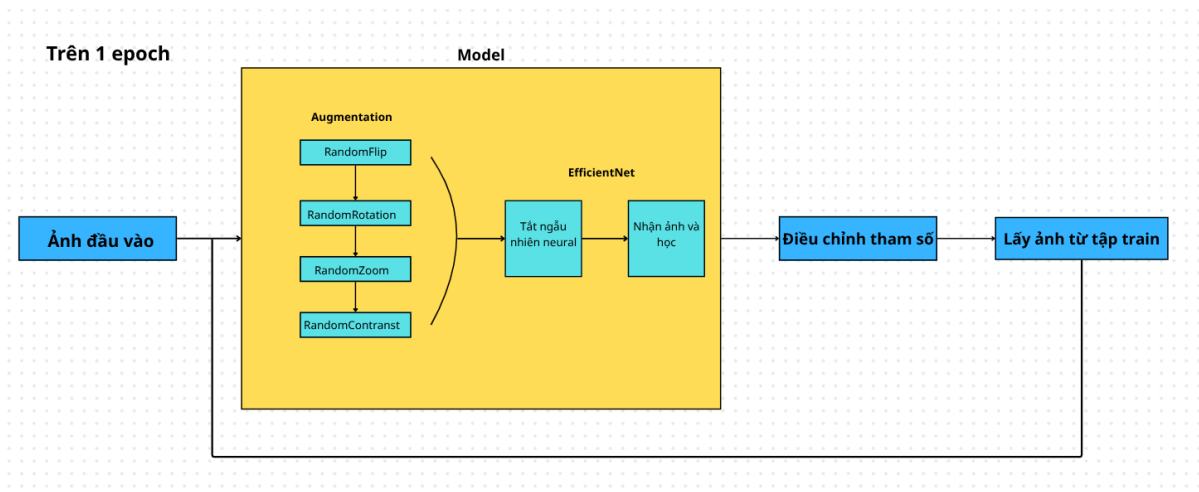
+OutputLayer:

Pipeline

Với mỗi ảnh đầu vào, nó sẽ bị biến đổi ngẫu nhiên qua 4 phép augmentation sau:

```
x = layers.RandomFlip("horizontal_and_vertical")(inputs)
x = layers.RandomRotation(0.2)(x) # Xoay ngẫu nhiên 20%
x = layers.RandomZoom(0.2)(x)     # Zoom ngẫu nhiên 10%
x = layers.RandomContrast(0.1)(x) # Thay đổi độ tương phản
```

sau đó đến phần head của EfficientNet, một tỉ lệ neural sẽ bị tắt và ảnh này sẽ được học, qua nhiều epoch, 1 ảnh bất kì trong tập train sẽ được model học qua nhiều góc độ hơn



-Sử dụng tối ưu hóa Adam, hàm mất mát CrossEntropy để điều chỉnh tham số, chạy trên 15 epoch

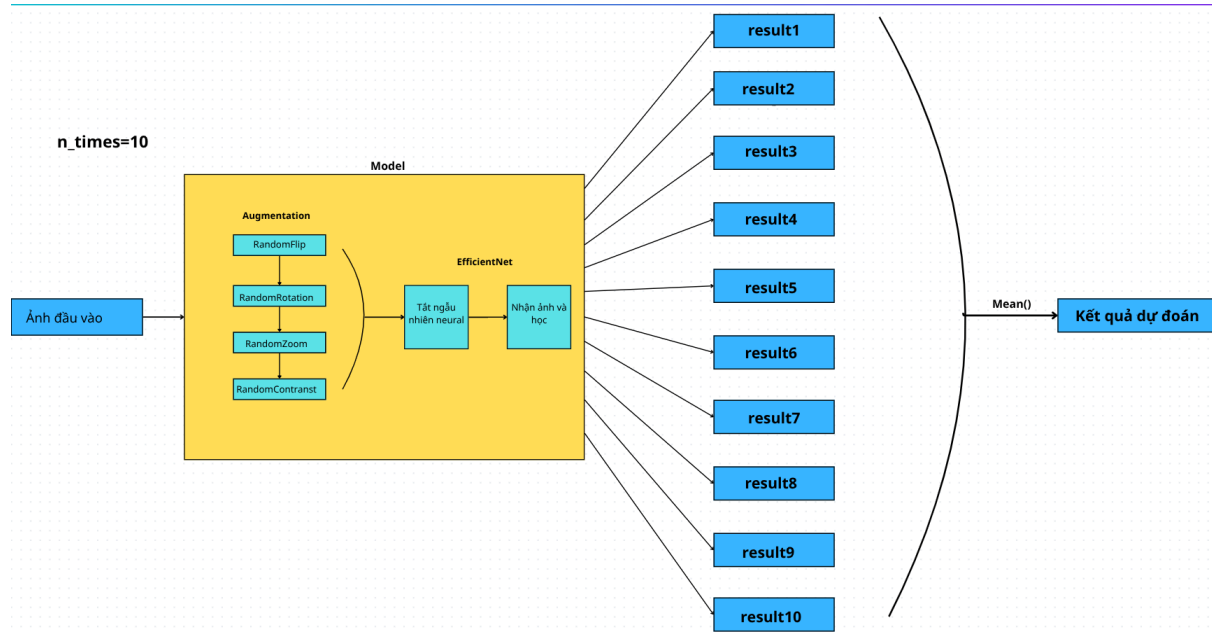
Fine-tuning: Sau khi model đã “hiểu” được với data mới, ta tiến hành fine-tuning, xả băng toàn bộ để model có thể học từ đầu, pipeline giống hệt như Training

Câu hỏi: Nếu không thực hiện transfer learning trước, mà tiến thẳng đến fine-tuning luôn thì kết quả sẽ như thế nào, giải thích

Câu hỏi: So sánh việc tạo ra các augmentation ngẫu nhiên so với việc tạo ra nhiều augmentation từ 1 ảnh

Prediction: -**TTA là gì:** TTA(Test time augmentation) là 1 kĩ thuật mà cùng một ảnh được biến đổi thành nhiều biến thể khác nhau thông qua các phép augmentation sau đó model sẽ dự đoán nhiều lần (bao nhiêu biến thể ảnh thì dự đoán bấy nhiêu lần), sau cùng là lấy trung bình kết quả

-**Prediction:** Khi đưa 1 ảnh vào model để dự đoán, trước hết model cũng sẽ biến đổi nó thành 1 biến thể khác để đem đi dự đoán, với $n_times=10$ nghĩa là với 1 bức ảnh model sẽ dự đoán 10 lần với 10 biến thể khác nhau, sau cùng nó sẽ tính trung bình của 10 lần dự đoán và cho ra kết quả cuối cùng



Chức năng của validation set: với mỗi epoch, model sẽ lấy data từ validation set để đưa ra các kết quả thống kê, để người dùng có thể nhìn thấy năng lực của model hiện tại, cũng như xác định earlyStop khi đã tối ưu hóa hết mức mà số lượng epoch cần phải chạy tiếp vẫn còn, giúp tiết kiệm được thời gian, bộ nhớ

Đức tập thuyết trình

Demo Hệ thống Web

Để kiểm chứng khả năng ứng dụng thực tế của mô hình trong bối cảnh hỗ trợ chẩn đoán lâm sàng, nhóm chúng em đã xây dựng một hệ thống Web Demo dựa trên ngôn ngữ Python, sử dụng framework Streamlit cho giao diện người dùng và thư viện TensorFlow để vận hành quy trình suy luận (inference) phía sau

Quy trình sử dụng hệ thống rất đơn giản với 3 bước là: Tải ảnh lên, Hệ thống phân tích và Trả về kết quả

Ở đây, khi người dùng tải ảnh lên, hệ thống sẽ tự động tiền xử lý và đưa qua mô hình **EfficientNet-B0** để dự đoán. Kết quả trả là tên bệnh (ví dụ: Ung thư tế bào đáy - BCC) + Mức độ cảnh báo + Độ tin cậy và có biểu đồ phân phối xác suất độ tin cậy (thư viện Altair)

Kết luận

- **Tổng kết:**
 - + Tổng kết lại đề án này, nhóm em đã xây dựng thành công quy trình phân loại tổn thương da sử dụng mô hình **EfficientNet-B0** kết hợp với **Transfer Learning** và **Fine-tuning** toàn mạng.
 - + Kết quả thực nghiệm cho thấy, nhờ áp dụng kỹ thuật suy luận nâng cao là **Test-Time Augmentation (TTA)** và **MC Dropout**, mô hình đã đạt độ chính xác 87.49% trên tập kiểm tra.
 - + Ưu điểm lớn nhất là mô hình gọn nhẹ, chi phí tính toán thấp, rất phù hợp để triển khai trên các thiết bị có tài nguyên hạn chế
- **Hạn chế và hướng phát triển:**

Bên cạnh những kết quả đạt được, nhóm cũng nhìn nhận thẳng thắn một số hạn chế còn tồn tại:

Về mất cân bằng lớp: Dù đã xử lý dữ liệu, sự nhầm lẫn vẫn xảy ra ở các lớp có đặc trưng hình ảnh chồng lấn nhau, điển hình là giữa Ung thư hắc tố (MEL) và Nốt ruồi lành tính (NV).

Về khả năng tổng quát hóa: Mô hình hiện tại chưa được kiểm chứng trên các nguồn dữ liệu độc lập khác ngoài HAM10000, nên khả năng hoạt động trong môi trường lâm sàng thực tế khác (cross-domain) cần được đánh giá thêm.

Về tính giải thích: Hiện tại hệ thống chưa có cơ chế minh họa vùng quyết định (như Heatmap), khiến việc thuyết phục bác sĩ tin vào dự đoán còn khó khăn.

→ Phát triển:

Từ đó, nhóm đề xuất các hướng phát triển trong tương lai

Tích hợp Siêu phân giải (Super-resolution): Để xử lý các ảnh đầu vào bị mờ hoặc độ phân giải thấp, giúp nâng cao chất lượng đặc trưng đầu vào.

Phát triển Explainable AI (XAI): Cụ thể là tích hợp **Grad-CAM** hoặc **Attention Map** để khoanh vùng tổn thương trên ảnh, giúp bác sĩ hiểu tại sao mô hình đưa ra dự đoán đó.

ì fi cìn nét bi